강의 자료 · 강사용 (명령어 주석 포함)

Git & GitHub, 진짜 쉽게

바이브 코더를 위한 터미널 Git · 단계별 용어 풀이와 명령어

add commit push



목차

들어가며	이 자료 읽는 법	3
시작하기	Git이 뭔데? 왜 터미널로 배워?	4
용어	어려운 용어 → 쉬운 말 번역표	7
한눈에 보기	전체 흐름 한 줄	9
1단계	터미널과 친해지기	11
2단계	Git 설치 & 첫 설정	16
3단계	저장소 만들기 (git init · git status)	19
4단계	세이브하기 (git add · git commit)	22
5단계	기록 보기 & 비교 (git log · git diff)	27
6단계	되돌리기 (타임머신)	30
7단계	올리면 안 되는 것 (.gitignore)	34
8단계	GitHub에 올리기 (remote · push · clone)	38
9단계	평행 세계 (branch · merge)	43
10단계	충돌 해결 (merge conflict)	47
11단계	합치기 신청서 (Pull Request)	50
12단계	같이 하기 (pull · Issues · 협업자 · fork)	55
13단계	내 사이트 공개하기 (GitHub Pages)	59
14단계	안전 수칙 (클로드에게 Git 맡기기 전에)	62
보너스	GitHub Actions 맛보기 (자동 검사 · 자동 배포)	66
부록 A	상황별 명령어 치트시트	68
부록 B	에러 메시지 번역표	70
부록 C	바이브 코더의 Git 10계명	72

들어가며

이 자료 읽는 법

🎤 **강사용**입니다. 학생용과 내용은 같고, 명령어마다 옆에 `# 설명` 주석이 달려 있습니다.

바이브 코딩하는 사람을 위한 **터미널 Git** 입문서입니다. 코드는 AI가 써도, **저장·되돌리기·올리기·합치기는 내가 이해하고 있어야** 합니다. 각 단계는 같은 순서로 되어 있습니다.

1. 📖 **이게 뭐야?** — 단계에 나오는 용어를 하나씩 풀어서 설명
2. ❓ **왜 필요해?** — 안 하면 무슨 일이 생기는지
3. ⌨️ **터미널에서 직접** — 실제로 치는 명령어와 화면 읽는 법
4. ⚠️ **자주 하는 실수**
5. 💬 **클로드에게 이렇게 말하기** — 복붙용 프롬프트

Git이 뭔데? 왜 터미널로 배워?

📖 이게 뭐야?

Git(깃) 은 **게임의 세이브 포인트**입니다. 보스전 앞에서 저장해두면, 져도 그 자리에서 다시 시작할 수 있죠. Git은 내 프로젝트 폴더를 **원하는 순간마다 통째로 저장**해두고, 언제든 그 시점으로 **되돌아갈 수 있게** 해줍니다.

GitHub(깃허브) 는 그 세이브 파일을 올려두는 **인터넷 창고**입니다. 컴퓨터가 고장 나도 남아 있고, 친구와 같이 작업하고, 웹사이트로 공개할 수도 있습니다.

	Git	GitHub
뭐야?	저장 기록을 관리하는 **프로그램**	Git 저장소를 올려두는 **웹사이트**
어디 있어?	내 컴퓨터	인터넷 (github.com)
비유	카메라 (사진 찍기)	인스타그램 (사진 올리고 공유)
인터넷 필요?	❌ 없어도 됨	✅ 필요



❓ 바이브 코딩인데 왜 Git이 필수야?

AI는 **한 번에 파일 20개를 고칩니다.** 결과가 마음에 안 들면?

- Git이 없으면 → "아까 그 버전으로 돌려줘" 라고 **AI에게 빌어야** 합니다. 똑같이 돌아왔는지 확신할 수 없습니다.
- Git이 있으면 → 명령어 한 줄로 **정확히** 그 시점으로 돌아갑니다.

> **세이브 없이 AI에게 큰 일을 시키는 건, 저장 안 하고 보스전 들어가는 것과 같습니다.**

❓ 클로드가 대신 쳐주는데 왜 터미널을 배워?

클로드 코드는 명령어를 실행하기 전에 **"이거 실행해도 돼요?"** 라고 묻습니다.

```
Bash command:  git reset --hard HEAD~3
Do you want to proceed?  ❯ 1. Yes   2. No
```

이게 무슨 뜻인지 모르면 **"Yes 누르는 기계"** 가 됩니다. (위 명령은 최근 커밋 3개와 저장 안 한 변경을 **영구 삭제**합니다.) 이 강의의 목표는 Git 전문가가 아니라, **AI가 치는 명령을 읽고 OK/NO를 판단할 수 있는 사람**이 되는 것입니다.

오늘 배우는 것	이유
명령어를 직접 쳐보기	손으로 쳐봐야 화면 메시지를 읽을 수 있음
화면 메시지 읽는 법	AI가 실패했을 때 무슨 일인지 알 수 있음
위험한 명령 구별	AI가 사고 치기 전에 NO 누르기
클로드에게 시키는 법	다 알고 나면, 그다음엔 맡겨도 됨

용어

어려운 용어 → 쉬운 말 번역표

원래 용어	쉬운 말	자세한 설명
터미널 / 셸	**글자로 컴퓨터에게 명령하는 창**	1단계
경로 (path)	**폴더 주소**	1단계
repository (저장소, repo)	**세이브 기록이 붙은 폴더**	3단계
`.git` 폴더	**블랙박스** (모든 기록이 여기에)	3단계
working directory	**내 책상** (지금 보고 있는 파일들)	4단계
staging area (index)	**택배 상자** (이번에 저장할 것 골라 담기)	4단계
commit	**세이브 포인트**	4단계
commit hash	**세이브 번호** (a1b2c3d)	4단계
HEAD	**"지금 여기" 지도 핀**	5단계
diff	**틀린 그림 찾기** (뭐가 바뀌었나)	5단계
restore / reset / revert	**되돌리기 3형제**	6단계

원래 용어	쉬운 말	자세한 설명
stash	서랍에 잠깐 치워두기	6단계
.gitignore	"이건 올리지 마" 목록	7단계
remote / origin	인터넷 창고 / 창고 별명	8단계
push / pull / clone	올리기 / 받아오기 / 통째로 복사	8, 12단계
branch	평행 세계	9단계
merge	평행 세계 합치기	9단계
conflict	"누구 걸로 할까?" 질문	10단계
Pull Request (PR)	"제 작업 합쳐도 될까요?" 신청서	11단계
Issue	할 일 카드	12단계
fork	남의 저장소를 내 계정으로 복사	12단계
GitHub Pages	무료 웹사이트 호스팅	13단계
force push	남의 기록까지 덮어쓰는 핵폭탄	14단계

한눈에 보기

전체 흐름 한 줄

```
터미널 익히기 → Git 설정 → 저장소 만들기 → 세이브(add·commit) → 기록 보기(log·diff)
→ 되돌리기 → 올리면 안 되는 것(.gitignore) → GitHub에 올리기(push)
→ 평행 세계(branch) → 충돌 해결 → 합치기 신청(PR) → 같이 하기(pull·issue) → 배포(Pages)
```

구간	단계	언제 쓰나
준비	1~2단계	컴퓨터마다 **한 번**
혼자 쓰는 Git	3~7단계	**매일, 수시로**
GitHub	8단계	프로젝트마다 **한 번** 연결, push는 수시로
같이 쓰는 Git	9~12단계	기능 하나마다 **반복**
공개	13단계	프로젝트마다 **한 번**
안전	14단계	**항상**

하루 작업 루틴 (이 순서만 외우면 됩니다)

```
git switch main && git pull       # main 으로 이동 후 GitHub 최신 커밋 받아오기 (&& = 앞이 성공하면 뒤 실행)
git switch -c feat/기능이름        # 새 브랜치 만들고 바로 이동 (-c = create)
#    ... 클로드와 작업 ...
git status                        # 지금 상태 확인 (아무것도 안 바꿈, 수시로 쳐도 안전)
git diff                          # add 전 변경을 줄 단위로 비교 (- 빨강 삭제 / + 초록 추가, q 로 나가기)
git add .                         # 현재 폴더의 바뀐 파일 전부 스테이징 (.gitignore 대상 제외)
git commit -m "feat: 기능 설명"    # 스테이징한 내용을 세이브 (-m = 메시지 바로 입력)
git push -u origin feat/기능이름   # 브랜치를 GitHub 에 처음 올림 (-u = 다음부터 git push 만)
gh pr create --fill               # PR 생성 (--fill = 커밋 메시지로 제목·본문 자동)
```

1단계

터미널과 친해지기

📖 이게 뭐야?

터미널 (Terminal) 마우스 대신 **글자로 컴퓨터에게 명령하는 창**입니다. 폴더 아이콘을 더블클릭하는 대신 `cd 폴더이름` 이라고 칩니다. 클로드 코드도 터미널 안에서 돌아가고, 클로드가 실행하는 명령도 전부 터미널 명령입니다.

셸 (Shell) 터미널 안에서 내 명령을 알아듣는 **통역사 프로그램**입니다. 종류가 여러 개입니다.

운영체제	추천	설명
Windows	**Git Bash**	Git 설치 시 같이 깔림. mac과 **같은 명령어**를 쓸 수 있어서 이 강의는 이걸 기준으로 합니다
Windows	PowerShell	`git` 명령은 똑같이 되지만 `ls`, `rm`, `touch` 등은 다르게 동작
mac	터미널 (zsh)	기본 앱

> VS Code 를 쓴다면: **Ctrl +** `**(백틱) 으로 터미널 열기 → 오른쪽 위 ∨ ` 메뉴에서** Git Bash** 선택

프롬프트 (Prompt) 명령을 기다리는 줄입니다. `$` 뒤에 명령을 칩니다. (이 자료에서 `$` 는 치지 않습니다)

```
nujabes@MY-PC MINGW64 ~/projects/my-site (main)
$ 
  └ 사용자@컴퓨터    └ 지금 위치(폴더)   └ Git 브랜치 (Git Bash가 알려줌)
```

명령어 읽는 법 — 명령은 대부분 **[프로그램] [하위 명령] [옵션] [대상]** 순서입니다.

```
git   commit   -m "feat: 메뉴 추가"
 |      |       └ 옵션(-m = message) + 값
 |      └ 하위 명령 (무엇을 할지)
 └ 프로그램 (누구에게 시킬지)
```

옵션은 짧게 `-m`, 길게 `--message` 두 가지 모양이 있는데 뜻은 같습니다.

경로 (Path) = 폴더 주소

기호	뜻	예
`~`	내 홈 폴더	`cd ~`
`.`	지금 이 폴더	`git add .` (여기 있는 전부)
`..`	한 칸 위 폴더	`cd ..`
`/`	폴더 구분	`cd projects/my-site`

⌨️ 터미널에서 직접 — 생존 명령어 10개

명령	뜻	기억법
`pwd`	지금 어디야?	print working directory
`ls`	여기 뭐 있어?	list
`ls -a`	숨긴 파일까지 (`.git` 보기)	all
`cd 폴더`	그 폴더로 이동	change directory
`cd ..`	위로 한 칸	
`mkdir 이름`	폴더 만들기	make directory
`touch 파일`	빈 파일 만들기	
`cat 파일`	파일 내용 보기	
`code .`	지금 폴더를 VS Code로 열기	
`clear`	화면 지우기	

꼭 알아야 할 키

키	하는 일
Tab	자동완성. `cd my-` 까지 치고 Tab → 나머지 자동 입력
↑ / ↓	아까 친 명령 다시 불러오기
Ctrl + C	실행 중인 것 **멈추기** (복사 아님!)
q	`git log` 같은 긴 화면에서 **나가기**

❓ 왜 필요해?

Git 명령은 **"지금 있는 폴더"** 에 대해 실행됩니다. 위치를 모르고 `git init` 을 치면 엉뚱한 폴더(심하면 홈 폴더 전체)가 저장소가 됩니다. `pwd` **로 위치 확인하는 습관**이 첫 번째 안전장치입니다.

⚠️ 자주 하는 실수

- 홈 폴더(`~`)에서 바로 `git init` → 컴퓨터 전체가 저장소가 됨
- 폴더 이름에 **띄어쓰기·한글** → `cd "my site"` 처럼 따옴표 필요. 처음엔 **영어 소문자 + 하이픈**(`my-site`) 추천
- `Ctrl + C` 를 복사로 쓰려다 프로그램이 꺼짐 → 터미널에서 복사는 **드래그 후 우클릭** 또는 Ctrl+Shift+C (터미널마다 다름)

💬 클로드에게 이렇게 말하기

```
방금 네가 실행한 터미널 명령을 한 줄씩, 각 부분(프로그램/하위명령/옵션/대상)이 무슨 뜻인지 설명해줘.
```

```
앞으로 터미널 명령을 실행하기 전에, 그 명령이 무엇을 바꾸는지 한 문장으로 먼저 말해줘.
```

2단계

Git 설치 & 첫 설정

📖 이게 뭐야?

git config Git에게 **"나는 누구고, 이렇게 동작해줘"** 를 알려주는 설정입니다. `--global` 을 붙이면 이 컴퓨터의 **모든 프로젝트**에 적용됩니다. 컴퓨터마다 **딱 한 번**만 하면 됩니다.

user.name / user.email 모든 세이브(커밋)에 **"누가 저장했나"** 도장이 찍힙니다. 이메일을 **GitHub 계정 이메일과 같게** 해야 GitHub에서 내 프로필(잔디)에 기록이 연결됩니다.

기본 브랜치 이름 (init.defaultBranch) 첫 평행 세계의 이름입니다. 예전엔 `master`, 지금 GitHub 기본값은 `main` 이라 맞춰줍니다.

⌨️ 터미널에서 직접

① 설치

```
# Windows (PowerShell 에서) — Git Bash 도 같이 깔립니다
winget install --id Git.Git -e  # Windows 에 Git + Git Bash 설치 (--id 패키지 지정, -e 정확히 일치)

# mac
xcode-select --install          # mac 개발자 도구 설치 (Git 포함)
```

② 설치 확인 (터미널을 **새로 열고**)

```
git --version  # 설치 확인 (안 나오면 터미널 껐다 켜기)
# git version 2.4x.x  ← 이렇게 나오면 성공
```

③ 첫 설정 (한 번만)

```
git config --global user.name "홍길동"  # 커밋에 찍힐 작성자 이름 (--global = 이 컴퓨터 전체)
git config --global user.email "github-가입-이메일@example.com"  # 커밋 작성자 이메일 (GitHub 가입 이메일과 같아야 잔디 연결)
git config --global init.defaultBranch main  # git init 시 첫 브랜치 이름을 master 대신 main 으로
git config --global core.editor "code --wait"  # 메시지 편집기를 VS Code 로 (--wait = 탭 닫을 때까지 대기, vim 감옥 방지)
git config --global pull.rebase false  # pull 할 때 merge 방식으로 합치기 고정 (divergent 경고 방지)
```

Windows 만 추가로:

```
git config --global core.autocrlf true  # Windows 줄바꿈(CRLF)↔LF 자동 변환 (mac 사용자와 diff 꼬임 방지)
```

④ 확인

```
git config --global --list  # 지금까지 한 전역 설정 전부 출력 (오타 확인)
```

❓ 왜 필요해?

설정 안 하고 커밋하면 이런 에러가 납니다.

```
Author identity unknown
*** Please tell me who you are.
```

= "누가 저장하는 건지 모르겠어요" → ③을 하면 해결됩니다.

⚠️ 자주 하는 실수

- 설치 직후 **원래 열려 있던 터미널**에서 `git --version` → "command not found". **터미널을 껐다 켜야** 새로 깐 프로그램을 인식합니다
- 이메일 오타 / GitHub과 다른 이메일 → 커밋은 되지만 GitHub 잔디가 안 심어짐
- 이메일을 공개하기 싫다면: GitHub → Settings → Emails 의 `숫자+아이디@users.noreply.github.com` 주소를 사용

💬 클로드에게 이렇게 말하기

```
git config --global --list 결과를 보고, Git 초보에게 필요한 기본 설정 중 빠진 게 있는지 확인해줘.
바꾸기 전에 어떤 명령으로 뭘 바꿀지 먼저 보여줘.
```

3단계

저장소 만들기 (git init · git status)

📖 이게 뭐야?

저장소 (Repository, 줄여서 repo) 세이브 기록이 붙은 폴더입니다. 겉보기엔 평범한 폴더인데, 안에 숨겨진 `.git` 폴더가 있습니다.

`.git` **폴더 = 블랙박스** 지금까지의 모든 세이브, 브랜치, 설정이 전부 여기 들어 있습니다. **이 폴더를 지우면 모든 기록이 사라집니다.** 그래서 숨김 처리되어 있고, 절대 손으로 건드리지 않습니다.

git init 평범한 폴더를 **저장소로 바꾸는 명령**입니다. 프로젝트마다 **처음 한 번**만 합니다.

git status "지금 상황 어때?" 라고 묻는 명령입니다. Git 명령 중 **가장 많이 치는 명령**이고, 아무것도 바꾸지 않으니 **몇 번을 쳐도 안전**합니다.

⌨️ 터미널에서 직접

```
cd ~                        # 홈 폴더로 이동 (~ = 내 홈)
mkdir projects              # 프로젝트 모아둘 폴더 만들기 (처음 한 번)
cd projects                 # projects 폴더로 들어가기
mkdir my-site && cd my-site # 폴더 만들고 바로 들어가기
pwd                         # 지금 위치 확인 (init 전에 꼭! 홈에서 init 사고 방지)
git init                    # 이 폴더를 저장소로 (.git 숨김 폴더 생성, 프로젝트당 한 번)
```

```
Initialized empty Git repository in /home/me/projects/my-site/.git/
```

```
ls -a       # 숨김 파일까지 목록 (-a = all, .git 생겼는지 확인)
git status  # 지금 상태 확인 (아무것도 안 바꿈, 수시로 쳐도 안전)
```

```
On branch main                          ← 지금 main 세계에 있음
No commits yet                          ← 아직 세이브 0개
nothing to commit (create/copy files and use "git add" to track)
```

git status 화면 읽는 법 (가장 중요!)

화면에 보이는 말	뜻	색
`Untracked files`	Git이 **처음 보는** 새 파일	🔴 빨강
`Changes not staged for commit`	바뀌었는데 아직 **상자에 안 담음**	🔴 빨강
`Changes to be committed`	**상자에 담김**, 커밋하면 저장됨	🟢 초록
`nothing to commit, working tree clean`	다 저장됨. **깨끗**	

> **빨강 = 아직 저장 대상 아님, 초록 = 다음 커밋에 들어감**

❓ 왜 필요해?

`git init` 을 하기 전에는 Git이 그 폴더를 **모릅니다.** 그 상태에서 git 명령을 치면:

```
fatal: not a git repository (or any of the parent directories): .git
```

= "여기 저장소 아닌데요?" → 위치가 맞는지 `pwd` 로 확인하고, 처음이면 `git init`.

⚠️ 자주 하는 실수

- 저장소 **안에서 또** `git init` (폴더 안의 폴더) → 기록이 꼬임. 하나의 프로젝트 = 하나의 `git init`
- `.git` 폴더가 거슬려서 삭제 → 모든 기록 증발
- 클로드 코드를 **다른 폴더에서** 실행 → 클로드가 엉뚱한 곳에 파일을 만듦. `cd my-site` 후 `claude`

💬 클로드에게 이렇게 말하기

```
git status 결과를 보고, 각 파일이 어떤 상태(새 파일/수정됨/커밋 대기)인지 표로 정리해줘.
```

4단계

세이브하기 (git add · git commit)

📖 이게 뭐야?

Git에는 **세 구역**이 있습니다. 이 그림이 Git의 전부라고 해도 됩니다.



스테이징 영역 (Staging Area) = 택배 상자 바로 저장하지 않고 **"이번 세이브에 넣을 것"을 골라 담는 곳**입니다. `git add 파일` = 상자에 담기. 상자에 담긴 것만 다음 커밋에 들어갑니다.

커밋 (Commit) = 세이브 포인트 상자 안의 내용을 **사진 찍듯 통째로 저장**합니다. 커밋 하나에는 다음이 같이 저장됩니다.

들어가는 것	예
파일 스냅샷	그 순간의 모든 파일 상태
메시지	`feat: 자기소개 섹션 추가`
작성자	홍길동 (hong@example.com)
시간	2026-10-12 10:30
해시 (세이브 번호)	`a1b2c3d` (앞 7글자만 써도 됨)

왜 상자(스테이징)가 따로 있어? 클로드가 한 번에 "헤더 디자인 변경 + 오타 수정 + 새 섹션" 을 했다면, **하나씩 나눠 커밋**할 수 있습니다. 나중에 "헤더 디자인만 되돌리기"가 가능해집니다.

⌨️ 터미널에서 직접

```
echo "<h1>안녕하세요</h1>" > index.html  # 글자를 index.html 에 저장 (> = 파일로 내보내기, 있으면 덮어씀)
git status                        # 지금 상태 확인 (아무것도 안 바꿈, 수시로 쳐도 안전)
git add index.html                # index.html 한 파일만 스테이징 (충돌 해결 후엔 '해결 완료' 표시)
git status                        # 지금 상태 확인 (아무것도 안 바꿈, 수시로 쳐도 안전)
git commit -m "feat: 첫 페이지 추가"  # 첫 세이브 (출력의 7글자 = 커밋 번호)
```

```
[main (root-commit) a1b2c3d] feat: 첫 페이지 추가
 1 file changed, 1 insertion(+)
 create mode 100644 index.html
```

`a1b2c3d` 가 이 세이브의 번호입니다.

자주 쓰는 add 모양

명령	뜻
`git add index.html`	이 파일만
`git add css/`	이 폴더 전체
`git add .`	지금 폴더 아래 **바뀐 것 전부** (가장 많이 씀, 대신 **status로 먼저 확인!**)

황금 습관: status → add → status → commit `add` 전후로 `git status` 를 쳐서 **빨강이 초록으로 바뀌는 걸 눈으로 확인**합니다.

좋은 커밋 메시지 = `타입: 무엇을`

타입	언제	예
`feat`	새 기능	`feat: 다크모드 버튼 추가`
`fix`	버그 수정	`fix: 모바일에서 메뉴 안 열리는 문제`
`style`	디자인·서식만	`style: 버튼 색 변경`
`docs`	문서	`docs: README 사용법 추가`
`refactor`	동작 그대로, 구조만 정리	`refactor: 중복 CSS 정리`
`chore`	설정·잡일	`chore: .gitignore 추가`

> 나쁜 예: `수정` , `ㅇㅇ` , `final` , `진짜최종` → 일주일 뒤 아무도 모릅니다

언제 커밋해?

- ✅ **클로드에게 큰 일을 시키기 직전** (세이브 후 보스전)
- ✅ 화면에서 확인해보고 **잘 동작할 때마다**
- ✅ 한 커밋 = **한 가지 일**

❓ 왜 필요해?

커밋하지 않은 변경은 **세이브가 안 된 것**입니다. AI가 파일을 망쳐도 돌아갈 곳이 없습니다. 커밋을 자주 할수록 돌아갈 수 있는 지점이 많아집니다.

⚠️ 자주 하는 실수

- `git commit` 만 치고 `-m` 을 빼먹음 → 이상한 편집기 화면(vim)에 갇힘 → 탈출법: `Esc` → `:q!` → **Enter** (취소) 또는 `Esc` → `:wq` → Enter (저장). 2단계에서 `core.editor` 를 설정했다면 VS Code가 열립니다
- add 없이 commit → `nothing added to commit but untracked files present`
- 확인 없이 `git add .` → 비밀번호 파일(`.env`)까지 담김 (7단계에서 해결)
- 클로드가 일한 **며칠치를 커밋 하나**로 → 되돌릴 때 전부 아니면 전무

💬 클로드에게 이렇게 말하기

```
지금 변경사항을 기능별로 나눠서 커밋하고 싶어.
어떤 파일들을 어떤 메시지로 묶을지 먼저 표로 보여주고, 내가 OK 하면 커밋해줘.
메시지는 "타입: 설명" 형식으로.
```

```
지금 작업 시작하기 전에 현재 상태를 커밋해서 세이브 포인트를 만들어줘.
```

5단계

기록 보기 & 비교 (git log · git diff)

📖 이게 뭐야?

git log = 세이브 목록 지금까지 저장한 커밋을 **최신순**으로 보여줍니다.

HEAD = "지금 여기" 지도 핀 지금 내가 보고 있는 커밋을 가리키는 표시입니다. 보통 가장 최근 커밋에 꽂혀 있습니다. `HEAD~1` = 하나 전, `HEAD~3` = 세 개 전.

git diff = 틀린 그림 찾기 무엇이 어떻게 바뀌었는지 줄 단위로 보여줍니다. 바이브 코딩에서 **가장 중요한 검수 도구**입니다.

⌨️ 터미널에서 직접

기록 보기

```
git log                         # 커밋 기록 자세히 (q 로 나가기)
git log --oneline               # 커밋 기록 한 줄씩 (짧은 번호 + 메시지)
git log --oneline --graph --all  # 브랜치 갈래까지 그림으로 (--all = 모든 브랜치)
git log --oneline -5            # 최근 5개만
```

```
c3d4e5f (HEAD -> main) style: 버튼 색 변경
b2c3d4e feat: 자기소개 섹션 추가
a1b2c3d feat: 첫 페이지 추가
```

바뀐 것 보기

명령	무엇을 비교?
`git diff`	작업 폴더 vs 마지막 저장 (**아직 add 안 한 것**)
`git diff --staged`	상자 안 vs 마지막 저장 (**add 한 것**)
`git diff 파일이름`	그 파일만
`git show a1b2c3d`	그 커밋에서 뭘 바꿨나

diff 화면 읽는 법

```
diff --git a/index.html b/index.html
@@ -3,7 +3,7 @@                  ← 3번째 줄 근처가 바뀜
   <body>
-    <h1>안녕하세요</h1>            ← 빨강 - : 지워진 줄
+    <h1>반갑습니다!</h1>           ← 초록 + : 새로 생긴 줄
   </body>
```

❓ 왜 필요해?

클로드가 "다 했어요!" 라고 해도, **실제로 뭘 바꿨는지는 diff만 정확히 알려줍니다.**

- 시키지 않은 파일을 건드렸나?
- 멀쩡한 코드를 지웠나?
- 비밀번호를 코드에 박아 넣었나?

> **AI 작업 검수 =** `git diff` **.** 커밋하기 전에 매번 봅니다.

⚠️ 자주 하는 실수

- `git log` 화면에서 못 나옴 → `q`
- `git add` 한 뒤에 `git diff` → 아무것도 안 나옴 (이미 상자에 들어갔으니까) → `git diff --staged`
- diff가 너무 길면 → `git diff --stat` 으로 **파일별 바뀐 줄 수**만 먼저 보기

💬 클로드에게 이렇게 말하기

```
방금 네가 바꾼 걸 git diff --stat 으로 먼저 보여주고,
파일별로 무엇을 왜 바꿨는지 한 줄씩 요약해줘. 내가 요청하지 않은 변경이 있으면 따로 표시해줘.
```

```
git log --oneline -10 을 보고, 이 프로젝트가 어떤 순서로 만들어졌는지 설명해줘.
```

6단계

되돌리기 (타임머신)

📖 이게 뭐야?

되돌리기는 **"어디까지 진행됐느냐"** 에 따라 명령이 다릅니다. 이 표 하나만 기억하세요.

상황	명령	위험도
파일을 고쳤는데 add 전, 원래대로	`git restore 파일`	⚠️ 고친 내용 영구 삭제
add 했는데 상자에서 빼기	`git restore --staged 파일`	✅ 안전 (수정 내용은 남음)
방금 커밋 메시지 오타	`git commit --amend -m "새 메시지"`	✅ push 전이면 안전
방금 커밋 취소 (변경은 살리기)	`git reset --soft HEAD~1`	✅ push 전이면 안전
이미 push 한 커밋 취소	`git revert 커밋번호`	✅ 가장 안전 (취소 커밋을 새로 만듦)
하던 거 잠깐 치워두기	`git stash` → 나중에 `git stash pop`	✅ 안전
옛날 버전 파일 하나만 가져오기	`git restore --source=커밋번호 파일`	⚠️ 지금 내용 덮어씀
커밋 안 한 거 전부 버리기	`git reset --hard`	💀 되돌릴 수 없음

reset vs revert — 제일 헷갈리는 둘

```
reset  = 기록을 지우고 과거로 돌아감        (내 공책에서 페이지 찢기)
revert = "앞의 걸 취소합니다" 새 기록을 추가  (공책에 "↑ 취소" 라고 새로 쓰기)
```

- 나 혼자 보는 기록(push 전) → `reset` 써도 됨
- 남도 보는 기록(push 후) → **반드시** `revert`. 찢어버리면 친구 공책과 안 맞습니다

stash = 서랍 하던 일을 커밋하긴 애매한데 급히 다른 브랜치로 가야 할 때, **서랍에 잠깐 넣어두는 것**입니다.

⌨️ 터미널에서 직접 — 🔥 "AI가 망쳤다" 구조 레시피

```
# 1. 진정하고 상황 파악
git status                  # 지금 상태 확인 (아무것도 안 바꿈, 수시로 쳐도 안전)
git diff --stat             # 파일별 바뀐 줄 수 요약 (AI 작업 후 첫 확인)

# 2-A. 아직 커밋 안 했다 → 마지막 세이브로
git restore .               # add 안 한 수정 전부 마지막 커밋으로 되돌림 (복구 불가, 새 파일은 안 지움)
git clean -n                # 지워질 새 파일 목록 미리보기만 (-n = dry run, 실제로 안 지움)
git clean -fd               # Git 이 모르는 새 파일(-f)·폴더(-d) 진짜 삭제 (.gitignore 대상은 안 건드림)

# 2-B. 이미 커밋까지 했다 (push 전)
git log --oneline -5        # 최근 5개만
git reset --soft HEAD~1     # 마지막 커밋만 취소, 변경은 스테이징에 남김 (HEAD~1 = 한 개 전, push 전만)

# 2-C. 이미 push 했다
git revert --no-edit a1b2c3d  # 그 커밋을 반대로 되돌리는 새 커밋 생성 (기록 유지, push 후에도 안전)
git push                    # 새 커밋을 GitHub 에 올리기 (-u 설정 후엔 이것만)
```

> 💡 클로드 코드에도 **되감기**(`Esc` 두 번 또는 `/rewind`) 기능이 있습니다. 하지만 **그 대화 안에서 클로드가 직접 고친 파일만** 되돌립니다. 확실한 세이브는 Git입니다.

❓ 왜 필요해?

되돌리는 법을 알아야 **AI에게 과감하게 시킬 수 있습니다.** "망쳐도 1분이면 복구" 라는 믿음이 바이브 코딩의 속도를 만듭니다.

⚠️ 자주 하는 실수

- `git restore`, `git reset --hard`, `git clean -fd` 는 **휴지통이 없습니다.** 커밋 안 한 내용은 진짜로 사라집니다
- **push 한 뒤에** `reset` 이나 `--amend` → 다시 올리려면 force push가 필요해짐 → 팀 사고 (14단계)
- `git clean -fd` 를 미리보기(`-n`) 없이 바로 실행

📦 더 알고 싶은 사람만 (몰라도 실습 가능)

reflog = 최후의 구조대. `reset --hard` 로 커밋을 날렸어도, **커밋했던 적이 있다면** 30일 정도는 `git reflog` 에 흔적이 남습니다. 거기서 번호를 찾아 `git reset --hard 번호` 로 복구할 수 있습니다. 단, **한 번도 커밋 안 한 변경은 reflog에도 없습니다.**

옛날 버전 구경만 하기: `git switch --detach 커밋번호` → 그 시점 파일들로 바뀜 (`detached HEAD` 경고는 "지금 과거를 구경 중" 이라는 뜻) → 다 봤으면 `git switch main`.

💬 클로드에게 이렇게 말하기

```
커밋하지 않은 변경을 전부 되돌리고 싶어.
실행하기 전에 되돌려질 파일과 삭제될 새 파일 목록을 먼저 보여주고, 내가 OK 하면 실행해.
```

```
이미 push 한 커밋 a1b2c3d 를 취소하고 싶어. reset 말고 revert 로 해줘.
```

7단계

올리면 안 되는 것 (.gitignore)

📖 이게 뭐야?

.gitignore = "이건 올리지 마" 목록 프로젝트 맨 위에 두는 텍스트 파일입니다. 여기 적힌 파일·폴더는 Git이 **못 본 척**합니다. `git add .` 를 해도 안 담깁니다.

절대 올리면 안 되는 것

대상	이유
`.env`	**API 키·비밀번호.** 공개 저장소에 올라가면 몇 분 안에 봇이 훔쳐 갑니다
`node_modules/`	`npm install` 로 언제든 다시 생김. 파일 수만 개
`dist/` , `build/`	빌드 결과물. 다시 만들 수 있음
`.DS_Store` , `Thumbs.db`	운영체제가 만드는 쓰레기 파일
`*.log`	로그 파일

⌨️ 터미널에서 직접

`.gitignore` 예시:

```
# 비밀
.env
.env.*
!.env.example

# 다시 만들 수 있는 것
node_modules/
dist/
build/

# 운영체제 / 에디터
.DS_Store
Thumbs.db
*.log
```

- `#` 은 메모, `/` 로 끝나면 폴더, `*` 는 아무 글자, `!` 는 "이건 예외로 올려"
- `.env.example` = 키 **이름만** 적은 견본 (`API_KEY=여기에_키`). 이건 올려서 팀원에게 "이런 키가 필요해" 를 알려줍니다

이미 올라간 파일을 나중에 ignore 에 추가하면? → 계속 추적됩니다. 추적만 끊어야 합니다.

```
git rm --cached .env             # Git 추적만 중단, 내 컴퓨터 파일은 유지 (--cached 빼면 파일도 삭제!)
git commit -m "chore: .env 추적 중단"  # 추적 중단 세이브 (예전 커밋엔 남아 있음 → 키 폐기 먼저)
```

❓ 왜 필요해?

클로드는 기능을 만들다가 **API 키를 `.env` 에 넣어두는 경우가 많습니다.** `.gitignore` 없이 `git add .` → `git push` 하면 **전 세계에 키를 공개**하게 됩니다. 요금 폭탄이나 계정 도용으로 이어집니다.

🚨 이미 키를 올려버렸다면

1. **키부터 폐기하고 새로 발급** (가장 먼저! 그 서비스 사이트에서)
2. 그다음 `git rm --cached` + `.gitignore` 추가
3. ⚠️ 파일을 지우고 커밋해도 **이전 커밋 기록에는 키가 남아 있습니다.** 그래서 1번이 핵심입니다

> GitHub은 유명한 키 모양을 발견하면 push를 막기도 합니다(Push Protection). 막혔다면 **고마운 일**입니다. 우회하지 마세요.

⚠️ 자주 하는 실수

- `.gitignore` 를 **첫 커밋 후에** 만듦 → 이미 들어간 파일은 계속 추적됨
- 파일 이름 오타: `gitignore` (점 없음), `.gitignore.txt` (Windows 메모장이 확장자 붙임)
- 비밀번호를 코드 파일(`app.js`)에 직접 적음 → ignore 로 못 막습니다. 키는 무조건 `.env` 로

💬 클로드에게 이렇게 말하기

```
이 프로젝트에 맞는 .gitignore 를 만들어줘. .env 는 꼭 넣고, .env.example 은 올라가게 해줘.
그리고 git status 로 지금 추적 중인 파일 중에 올리면 안 되는 게 있는지 검사해줘.
```

```
앞으로 API 키나 비밀번호는 절대 코드에 직접 쓰지 말고 .env 에 넣어. CLAUDE.md 에도 이 규칙을 적어줘.
```

8단계

GitHub에 올리기 (remote · push · clone)

📖 이게 뭐야?

원격 저장소 (Remote) GitHub 위에 있는 **내 저장소의 쌍둥이**입니다. 내 컴퓨터 저장소 = **로컬(local)**, GitHub 저장소 = **원격(remote)**.

origin 원격 저장소 주소에 붙이는 **별명**입니다. `https://github.com/hong/my-site.git` 을 매번 칠 수 없으니 `origin` 이라고 부릅니다. (관례상 거의 다 origin)

명령	뜻	방향
`git push`	내 커밋을 GitHub에 **올리기**	로컬 → GitHub
`git pull`	GitHub의 새 커밋을 **받아오기**	GitHub → 로컬
`git clone`	GitHub 저장소를 **통째로 복사**해 오기 (처음 한 번)	GitHub → 새 폴더

인증 (로그인) GitHub은 2021년부터 **비밀번호로 push 하는 걸 막았습니다.** 대신 아래 중 하나를 씁니다.

방법	난이도	설명
gh CLI (`gh auth login`)	⭐ 추천	브라우저로 로그인 한 번이면 끝. 이슈·PR 도 터미널에서 가능
토큰 (PAT)	⭐⭐	비밀번호 대신 쓰는 긴 문자열. 유출 주의
SSH 키	⭐⭐⭐	열쇠 파일을 만들어 등록. 익숙해지면 편함

gh = GitHub CLI GitHub 웹사이트에서 클릭하던 일(저장소 만들기, PR, 이슈)을 **터미널에서 하게 해주는 공식 도구**입니다. 클로드 코드도 GitHub 작업을 할 때 주로 이걸 씁니다.

⌨️ 터미널에서 직접

① gh 설치 & 로그인 (컴퓨터마다 한 번)

```
# Windows (PowerShell)
winget install --id GitHub.cli  # Windows 에 GitHub CLI(gh) 설치 (설치 후 터미널 재시작)
# mac
brew install gh                 # mac 에 GitHub CLI 설치
```

터미널을 **새로 열고**:

```
gh auth login  # GitHub 로그인 (HTTPS + Yes 고르면 git push 인증까지 자동)
```

질문에 이렇게 답합니다.

```
? Where do you use GitHub?            → GitHub.com
? What is your preferred protocol?     → HTTPS
? Authenticate Git with your GitHub credentials? → Yes
? How would you like to authenticate?  → Login with a web browser
! First copy your one-time code: ABCD-1234   ← 이 코드를 브라우저에 입력
```

```
gh auth status  # 로그인 상태 확인 (✓ Logged in 이면 성공)
```

② 방법 A — gh 로 한 방에 (추천)

```
gh repo create my-site --public --source=. --remote=origin --push  # 공개 저장소 생성 + 현재 폴더 연결 + origin 등록 + 업로드를 한 번에
```

= GitHub에 `my-site` 저장소를 만들고(`create`), 지금 폴더(`--source=.`)를 연결하고(`--remote=origin`), 커밋을 올림(`--push`)

② 방법 B — 웹에서 만들고 연결

1. github.com → **New repository** → 이름 입력 → ⚠️ **README·.gitignore 체크하지 말고** Create
2. 화면에 나온 명령을 터미널에:

```
git remote add origin https://github.com/아이디/my-site.git  # GitHub 주소를 origin 이라는 별명으로 등록 (연결만)
git branch -M main               # 현재 브랜치 이름을 main 으로 강제 변경 (이미 main 이면 그대로)
git push -u origin main          # main 첫 업로드 + origin/main 과 짝꿍 설정
```

- `-u` = "앞으로 이 브랜치는 origin/main 과 짝꿍" → 다음부터는 `git push` **만** 쳐도 됨

③ 확인

```
git remote -v       # 연결된 원격 주소 확인 (-v = 자세히)
gh repo view --web  # 이 저장소를 브라우저로 열기
```

④ 그다음부터는 이것만

```
git add .                       # 현재 폴더의 바뀐 파일 전부 스테이징 (.gitignore 대상 제외)
git commit -m "feat: 연락처 섹션 추가"  # 평소 작업 세이브
git push                        # 새 커밋을 GitHub 에 올리기 (-u 설정 후엔 이것만)
```

clone — 다른 컴퓨터에서 / 남의 프로젝트 받기

```
cd ~/projects                   # 프로젝트 폴더로 (clone 은 저장소 바깥에서)
git clone https://github.com/아이디/my-site.git  # GitHub 저장소 통째로 복사 (폴더 + 기록 + origin 자동 연결)
cd my-site                      # 복사된 폴더로 들어가기
npm install                     # package.json 보고 node_modules 다시 설치 (Node 프로젝트만)
```

❓ 왜 필요해?

- 내 컴퓨터가 고장 나도 **코드가 살아남음** (백업)
- 집·학원 컴퓨터 어디서든 `clone` / `pull` 로 이어서 작업
- 포트폴리오: GitHub 프로필이 곧 **개발 이력서**
- 협업·배포(9~13단계)의 출발점

⚠️ 자주 하는 실수 — 에러 메시지 번역

에러	번역	해결
`src refspec main does not match any`	올릴 커밋이 없어요	먼저 `git add` + `git commit`
`Permission denied` / `403` / `Authentication failed`	로그인 안 됐거나 권한 없음	`gh auth login`, 내 저장소가 맞는지 확인
`! [rejected] main -> main (fetch first)`	GitHub에 내가 없는 커밋이 있어요	`git pull` 후 다시 push (12단계)
`remote origin already exists`	origin 이 이미 있어요	`git remote -v` 로 확인. 바꾸려면 `git remote set-url origin 새주소`

- 웹에서 저장소 만들 때 **README 를 체크**해버림 → 첫 push가 `rejected` → `git pull --rebase origin main` 한 번 하고 `git push -u origin main`
- 토큰을 채팅·코드에 붙여넣음 → 유출. 토큰은 **비밀번호와 똑같이** 다룹니다

💬 클로드에게 이렇게 말하기

```
gh 로 이 폴더를 GitHub 에 public 저장소 "my-site" 로 올려줘.
실행할 명령을 먼저 보여주고, 내가 OK 하면 실행해. 끝나면 저장소 주소를 알려줘.
```

```
git push 가 실패했어. 에러 메시지를 해석해주고, 해결 방법을 위험한 것 없이 알려줘.
```

9단계

평행 세계 (branch · merge)

📖 이게 뭐야?

브랜치 (Branch) = 평행 세계 지금 프로젝트에서 **갈라져 나온 복사 세계**입니다. 그 세계에서 무슨 짓을 해도 원래 세계(`main`)는 멀쩡합니다.



main 정식 버전입니다. 배포되는 것, 손님이 보는 것. 원칙: **main에서 직접 작업하지 않는다.**

merge = 평행 세계 합치기 실험이 성공하면 그 세계의 변경을 main 에 **합칩니다.** 실패하면? **브랜치째 버리면** 끝입니다.

브랜치는 파일을 통째로 복사하는 게 아니라 **책갈피 하나** 꽂는 것이라, 수백 개를 만들어도 가볍습니다.

브랜치 이름 짓기 — `종류/짧은-설명` (영어 소문자, 하이픈)

예	뜻
`feat/dark-mode`	새 기능
`fix/mobile-menu`	버그 수정
`docs/readme`	문서
`feat/3-contact-form`	이슈 3번 작업 (12단계)

⌨️ 터미널에서 직접

```
git branch                  # 브랜치 목록 (* = 지금 위치)
git switch -c feat/dark-mode  # 브랜치 만들고 이동 (= 옛 명령 git checkout -b)
#   ... 작업 & 커밋 ...
git switch main             # main 으로 이동 (폴더 파일이 main 상태로 바뀜)
git merge feat/dark-mode    # 지금 브랜치(main)에 feat/dark-mode 합치기
git branch -d feat/dark-mode  # 합친 브랜치 삭제 (-d = 안 합쳤으면 거절하는 안전 삭제)
```

GitHub 에도 브랜치 올리기:

```
git push -u origin feat/dark-mode  # 브랜치 GitHub 에 처음 올리기
```

실험 실패, 버리기:

```
git switch main              # main 으로 이동 (폴더 파일이 main 상태로 바뀜)
git branch -D feat/dark-mode  # 강제 삭제 (-D = 안 합친 커밋도 버림)
```

> 클로드가 `git checkout -b 이름` 을 쓸 수도 있습니다. **옛날 방식**이고 `git switch -c 이름` 과 같은 뜻입니다.

❓ 왜 필요해?

바이브 코딩의 최고 습관: **"클로드에게 큰 일 시키기 전에 브랜치 먼저."**

- 결과가 좋으면 → merge
- 망했으면 → 브랜치 삭제. main 은 아무 일도 없었던 것처럼 멀쩡
- 아이디어 두 개를 **두 브랜치에서 각각** 시켜보고 좋은 쪽만 합칠 수도 있음

⚠️ 자주 하는 실수

- 커밋 안 한 변경을 들고 `switch` → 이런 에러: `error: Your local changes to the following files would be overwritten by checkout` → 먼저 커밋하거나 `git stash` 후 이동
- 지금 어느 브랜치인지 모르고 작업 → **Git Bash 프롬프트의** `(브랜치이름)` 이나 `git branch` 로 항상 확인
- `git merge` 했더니 편집기(VS Code 탭)가 열림 → 합치기 커밋 메시지를 확인하라는 것. **저장하고 탭을 닫으면** 계속 진행됩니다
- main 에서 바로 클로드에게 큰 작업 → 되돌리기 번거로움

💬 클로드에게 이렇게 말하기

```
새 브랜치 feat/dark-mode 를 만들고 거기서 다크모드 토글을 추가해줘.
main 브랜치는 절대 건드리지 마. 끝나면 커밋하고 브랜치 이름을 알려줘.
```

```
지금 브랜치가 뭐고, main 과 비교해서 어떤 커밋이 추가됐는지 보여줘. (git log main..HEAD --oneline)
```

10단계

충돌 해결 (merge conflict)

📖 이게 뭐야?

충돌 (Conflict) 두 평행 세계에서 **같은 파일의 같은 줄을 서로 다르게** 고쳤을 때, Git이 **"둘 중 누구 걸로 할까요?"** 라고 묻는 것입니다. **에러가 아니라 질문**입니다. 무서워할 필요 없습니다.

다른 줄을 고쳤다면 Git이 **알아서 둘 다 합쳐줍니다.** 같은 줄일 때만 사람에게 묻습니다.

충돌 표시 읽는 법

```
<<<<<<< HEAD
<h1>홍길동의 홈페이지</h1>          ← 지금 내 쪽 (합치는 대상, 보통 main)
=======
<h1>길동이네 놀이터</h1>            ← 들어오는 쪽 (합치려는 브랜치)
>>>>>>> feat/new-title
```

`<<<<<<<`, `=======`, `>>>>>>>` 세 줄은 Git이 넣은 **표시**이고, 그 사이가 양쪽 내용입니다.

⌨️ 터미널에서 직접

```
git merge feat/new-title  # 같은 줄을 고친 브랜치 합치기 → 충돌 발생
```

```
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

해결 4단계

```
git status                      # 지금 상태 확인 (아무것도 안 바꿈, 수시로 쳐도 안전)
code index.html                 # 충돌 파일 VS Code 로 열기 (<<<<<<< ======= >>>>>>> 정리)
git add index.html              # index.html 한 파일만 스테이징 (충돌 해결 후엔 '해결 완료' 표시)
git commit -m "merge: 제목 충돌 해결"  # 합치기 마무리 커밋
```

②의 결과 예 (둘 중 하나, 또는 둘을 섞어도 됨):

```
<h1>길동이네 놀이터</h1>
```

VS Code 를 쓰면 충돌 위에 버튼이 뜹니다. `Accept Current` (내 쪽) / `Accept Incoming` (들어오는 쪽) / `Accept Both` (둘 다)

다 모르겠으면 — 합치기 취소

```
git merge --abort  # 합치기 취소, merge 전 상태로 복귀
```

❓ 왜 필요해?

혼자 해도 충돌은 납니다. (브랜치 두 개에서 클로드에게 같은 파일을 고치게 하면) 같이 하면 **반드시** 납니다. 충돌을 피하는 게 아니라 **침착하게 푸는 법**을 아는 게 중요합니다.

⚠️ 자주 하는 실수

- 표시(`<<<<<<<` 등)를 **안 지우고** 커밋 → 사이트에 그대로 `<<<<<<< HEAD` 가 보임 → 커밋 전에 `git diff` 로 표시가 남았는지 확인
- 충돌 났다고 폴더를 지우고 다시 clone → 내 작업 날아감. `git merge --abort` 가 정답
- 오래 혼자 작업하다 한 번에 합치기 → 충돌 폭탄. **작게, 자주** 합치기

💬 클로드에게 이렇게 말하기

```
merge 충돌이 났어. 충돌 난 파일과 각 충돌 부분에서 양쪽이 무엇을 바꾸려 했는지 설명하고,
어떻게 합칠지 추천해줘. 내가 고르면 그대로 해결하고, 충돌 표시가 남았는지 확인한 뒤 커밋해줘.
```

11단계

합치기 신청서 (Pull Request)

📖 이게 뭐야?

Pull Request (PR) = "제 작업 합쳐도 될까요?" 신청서 브랜치를 main 에 합치기 전에 GitHub 에 올리는 **검토 요청서**입니다. 9단계에서는 터미널에서 바로 `git merge` 했지만, 실제 현업에서는 **거의 항상 PR 을 거쳐서** 합칩니다.

PR 페이지에서 할 수 있는 것:

기능	설명
Files changed	바뀐 내용을 diff 로 한눈에
코드 리뷰	줄마다 댓글, `Approve` (승인) / `Request changes` (수정 요청)
자동 검사	테스트·빌드가 통과하는지 (CI, 보너스)
기록	"왜 이렇게 바꿨나" 대화가 영원히 남음

혼자 해도 PR 을 쓰는 이유 클로드가 한 작업을 **GitHub 화면에서 diff 로 차분히 검토**하는 습관이 생기고, 나중에 "이 기능 언제 왜 넣었지?" 를 PR 로 찾을 수 있습니다.

합치는 방식 3가지 (처음엔 **Squash** 추천)

방식	결과
Create a merge commit	브랜치의 커밋 전부 + 합치기 커밋
Squash and merge	브랜치의 커밋들을 **하나로 뭉쳐서** main 에 → main 기록이 깔끔
Rebase and merge	커밋들을 main 끝에 한 줄로 이어 붙임

⌨️ 터미널에서 직접 — PR 한 바퀴

```
# 1. 브랜치에서 작업하고 올리기
git switch -c feat/contact       # 기능 브랜치 만들고 이동
#    ... 작업 & 커밋 ...
git push -u origin feat/contact  # 브랜치 올리기 (PR 만들려면 먼저 올라가 있어야 함)

# 2. PR 만들기
gh pr create --title "feat: 연락처 섹션 추가" --body "이메일/깃허브 링크 섹션 추가. Closes #3"  # PR 생성 (Closes #3 → 합치면 이슈 3번 자동 종료)
#   또는 질문에 답하면서:  gh pr create
#   또는 커밋 메시지로 자동:  gh pr create --fill

# 3. 확인 / 리뷰
gh pr list                       # 열린 PR 목록
gh pr view --web                 # 현재 브랜치 PR 을 브라우저로 (Files changed 탭)
gh pr checkout 5                 # PR 5번 브랜치를 내 컴퓨터로 받아 이동
gh pr review 5 --approve         # PR 5번 승인 (자기 PR 은 승인 불가)

# 4. 합치기 + 원격 브랜치 삭제
gh pr merge 5 --squash --delete-branch  # 커밋 하나로 뭉쳐 합치고 브랜치 삭제

# 5. 내 컴퓨터 정리 (중요!)
git switch main                  # main 으로 이동 (폴더 파일이 main 상태로 바뀜)
git pull                         # GitHub 최신 커밋 받아 합치기 (= fetch + merge)
git branch                       # 브랜치 목록 (* = 지금 위치)
```

PR 본문 마법의 단어 본문에 `Closes #3` (또는 `Fixes #3`) → PR 이 합쳐지면 **이슈 3번이 자동으로 닫힘** (12단계)

좋은 PR 본문 3줄

```
무엇을: 연락처 섹션 추가 (이메일, GitHub 링크)
왜: 방문자가 연락할 방법이 없었음
확인: 모바일/PC 화면에서 링크 클릭 확인. Closes #3
```

❓ 왜 필요해?

- **main 보호**: 검토 안 된 코드가 정식 버전에 들어가지 않음
- **두 번째 눈**: 클로드가 만든 코드를 짝꿍(또는 미래의 나)이 한 번 더 봄
- 13단계 배포와 연결하면 **"PR 합치기 = 사이트 업데이트"** 가 됨

⚠️ 자주 하는 실수

- PR 합친 뒤 **로컬 main 을 pull 안 함** → 다음 브랜치가 옛날 main 에서 시작 → 충돌
- PR 하나에 기능 5개 → 리뷰 불가. **PR 하나 = 기능 하나**
- 리뷰 없이 바로 자기 PR merge (팀이라면)
- squash 로 합친 브랜치를 `git branch -d` 로 지우면 `not fully merged` 라며 거절 → PR 이 **Merged** 인 걸 확인했으면 `git branch -D` 로 지워도 됩니다
- PR 을 연 뒤에도 같은 브랜치에 커밋하고 `git push` 하면 **PR 에 자동으로 추가**됩니다 (새 PR 만들 필요 없음)

💬 클로드에게 이렇게 말하기

```
지금 브랜치를 push 하고 main 으로 PR 을 만들어줘.
제목은 "타입: 설명" 형식, 본문은 무엇을/왜/어떻게 확인했는지 3줄 + "Closes #이슈번호".
```

```
PR 5번을 리뷰해줘. gh pr diff 5 로 변경을 보고, 버그 가능성/안 시킨 변경/비밀정보 노출을 체크해서 알려줘.
merge 는 하지 마.
```

12단계

같이 하기 (pull · Issues · 협업자 · fork)

📖 이게 뭐야?

git pull = 받아오기 GitHub 에 친구(또는 다른 컴퓨터의 나)가 올린 커밋을 **내 컴퓨터로 가져와 합치는 것**입니다. 정확히는 `fetch`(가져오기만) + `merge`(합치기) 두 동작을 한 번에 합니다.

명령	하는 일
`git fetch`	GitHub 소식만 **확인** (내 파일은 안 바뀜)
`git pull`	가져와서 **내 브랜치에 합침**

아침 루틴: 작업 시작 전 항상 `git switch main && git pull`

Issue = 할 일 카드 GitHub 저장소의 **할 일 / 버그 / 아이디어 게시판**입니다. 번호(`#3`)가 붙고, 담당자·라벨을 달 수 있습니다. PR 에서 `Closes #3` 으로 연결하면 **일이 끝날 때 자동으로 닫힙니다.**

협업자 (Collaborator) vs 포크 (Fork)

	협업자 초대	Fork
언제	**우리 팀** 프로젝트	**남의** 오픈소스에 기여
방법	주인이 Settings → Collaborators → 초대	남의 저장소를 **내 계정으로 복사**
push	원본 저장소에 직접 브랜치 push 가능	내 복사본에 push → 원본에 PR

⌨️ 터미널에서 직접

받아오기

```
git switch main  # main 으로 이동 (폴더 파일이 main 상태로 바뀜)
git pull         # GitHub 최신 커밋 받아 합치기 (= fetch + merge)
```

push 가 거절됐을 때 (친구가 먼저 올림)

```
 ! [rejected]        main -> main (fetch first)
hint: Updates were rejected because the remote contains work that you do not have locally.
```

```
git pull  # GitHub 최신 커밋 받아 합치기 (= fetch + merge)
git push  # 새 커밋을 GitHub 에 올리기 (-u 설정 후엔 이것만)
```

> ⛔ 이때 `git push --force` 는 **절대 금지.** 친구의 커밋을 지워버립니다.

Issues

```
gh issue create --title "연락처 섹션 추가" --body "이메일, GitHub 링크"  # 이슈 생성 (출력 URL 끝 숫자 = 이슈 번호)
gh issue list                    # 열린 이슈 목록
gh issue view 3                  # 이슈 3번 내용 보기
gh issue edit 3 --add-assignee @me  # 이슈 3번 담당자를 나로 (@me = 내 계정)
gh issue close 3                 # 이슈 3번 직접 닫기
```

이슈 → 브랜치 → PR 흐름

```
Issue #3 등록 → git switch -c feat/3-contact → 작업·커밋·push
→ gh pr create (본문: Closes #3) → 리뷰 → merge → 이슈 #3 자동 닫힘 ✅
```

협업자 초대: 저장소 → **Settings → Collaborators → Add people** → 친구 아이디 → 친구가 메일/알림에서 **수락**

Fork 해서 기여하기

```
gh repo fork 원주인/저장소 --clone  # 남의 저장소를 내 계정에 복사 + 내 컴퓨터로 clone
cd 저장소                          # clone 된 폴더로 이동
git switch -c fix/typo           # 수정용 브랜치 만들고 이동
#   ... 작업 & 커밋 & push ...
gh pr create                     # 질문에 답하며 PR 생성 (fork 에선 원본 저장소로 감)
```

README.md = 저장소 간판 저장소 첫 화면에 보이는 설명서입니다. **무엇인지 / 어떻게 실행하는지 / 사이트 주소** 정도는 적어둡니다.

❓ 왜 필요해?

혼자 하던 Git 이 **팀 도구**가 되는 단계입니다. 이슈로 할 일을 나누고, 브랜치로 동시에 작업하고, PR 로 합치면 **여러 명이 같은 프로젝트를 망가뜨리지 않고** 만들 수 있습니다.

⚠️ 자주 하는 실수

- 아침에 pull 안 하고 작업 시작 → 오후에 충돌 폭탄
- `git pull` 했더니 편집기가 열림 → 친구 커밋과 내 커밋을 합치는 **merge 메시지** 확인 창. 저장하고 닫으면 끝
- 두 사람이 **같은 이슈**를 동시에 작업 → 담당자(assignee) 지정으로 예방
- 협업자 초대만 하고 친구가 **수락 안 함** → push 시 `Permission denied`

💬 클로드에게 이렇게 말하기

```
issues.md 에 정리한 할 일들을 gh issue create 로 하나씩 등록해줘. 등록 전에 목록을 먼저 보여줘.
```

```
이슈 3번을 작업할 거야. main 을 최신으로 pull 하고, feat/3-이슈요약 브랜치를 만든 다음,
이슈 내용을 읽고 구현해줘. PR 본문에는 Closes #3 을 넣어줘.
```

13단계

내 사이트 공개하기 (GitHub Pages)

📖 이게 뭐야?

GitHub Pages GitHub 저장소의 HTML 파일을 **무료로 인터넷에 공개해주는 서비스**입니다. 서버를 빌릴 필요가 없습니다.

```
주소 모양:  https://내아이디.github.io/저장소이름/
```

정적 사이트 (Static Site) HTML·CSS·JS 파일만으로 된 사이트입니다. (로그인, DB 가 필요한 사이트는 Pages 로는 안 됩니다)

프로젝트 종류	Pages 에 올리는 법
순수 HTML/CSS/JS (`index.html` 이 맨 위에)	**설정만 켜면 끝** ← 이 강의
Vite·React 등 빌드가 필요한 것	빌드 결과(`dist`)를 올려야 함 → GitHub Actions 사용 (보너스)

⌨️ 켜는 법

웹에서 (추천)

1. 저장소 → **Settings → Pages**
2. Source: **Deploy from a branch**
3. Branch: `main` / 폴더: `/ (root)` → **Save**
4. 1~2분 기다린 뒤 새로고침 → 위에 **"Your site is live at ..."** 주소가 뜸

터미널에서 (gh)

```
gh api repos/{owner}/{repo}/pages -X POST -f "source[branch]=main" -f "source[path]=/"  # API 로 Pages 켜기 (main 브랜치 루트 폴더 배포)
gh api repos/{owner}/{repo}/pages --jq .html_url  # Pages 정보에서 사이트 주소만 뽑기
```

(`{owner}/{repo}` 는 그대로 치면 gh 가 지금 폴더의 저장소로 채워줍니다)

그다음부터는 push = 배포

```
git add . && git commit -m "style: 메인 색상 변경" && git push  # 담기→세이브→올리기 한 줄로 (1~2분 뒤 사이트 반영)
# 1~2분 뒤 사이트에 반영. 저장소의 Actions 탭에서 배포 진행 상황 확인 가능
```

❓ 왜 필요해?

- 만든 걸 **링크 하나로** 친구·면접관에게 보여줄 수 있음
- "PR 합치기 → main 변경 → 사이트 자동 갱신" 흐름을 체감 = 실제 회사의 배포 흐름과 같은 구조

⚠️ 자주 하는 실수

- **404** 가 뜬다 →
  - `index.html` 이 저장소 **맨 위**에 있는지 (`src/index.html` 이면 안 됨)
  - 파일 이름 대소문자 (`Index.html` ❌)
  - 아직 1~2분 안 지남
- 이미지·CSS 가 안 나온다 → 경로가 `/images/a.png` (맨 앞 `/`) 이면 `https://아이디.github.io/images/...` 를 찾음 → `images/a.png` **처럼 상대 경로**로
- 무료 계정은 **public 저장소만** Pages 가능
- 공개 사이트이므로 **개인정보(전화번호, 주소)** 는 올리지 않기

💬 클로드에게 이렇게 말하기

```
이 사이트를 GitHub Pages 로 배포하려고 해. index.html 이 맨 위에 있는지,
이미지/CSS 경로 중 Pages 에서 깨질 절대경로(/로 시작)가 있는지 검사하고 고쳐줘.
```

```
README.md 를 만들어줘. 사이트 소개 한 줄, 배포 주소, 로컬에서 보는 방법을 넣어줘.
```

14단계

안전 수칙 (클로드에게 Git 맡기기 전에)

📖 이게 뭐야?

여기까지 왔다면 이제 **클로드에게 Git 작업을 맡겨도 됩니다.** 단, 아래 명령이 보이면 **무조건 멈추고 "왜?" 라고 물어보세요.**

💀 위험 명령 목록

명령	무슨 일이 생기나	대신 쓸 것
`git push --force` / `-f`	GitHub 의 기록을 **내 걸로 덮어씀** → 친구 커밋 삭제	`git pull` 후 `git push`
`git reset --hard`	커밋 안 한 변경 **영구 삭제**	`git stash` (서랍에 보관)
`git clean -fd`	새로 만든 파일 **영구 삭제**	먼저 `git clean -n` 미리보기
`git branch -D`	합치지 않은 브랜치 **강제 삭제**	`-d` (안전 버전)
`git checkout -- .` / `git restore .`	수정한 내용 **전부 원래대로**	먼저 `git diff` 확인
`git rebase` **(공유 브랜치에서)**	남들과 공유한 기록을 **다시 씀**	`git merge`
`rm -rf .git`	**모든 기록 증발**	절대 하지 않음

브랜치 보호 규칙 (Branch protection) GitHub 저장소 설정으로 **"main 에는 PR 로만 합칠 수 있다"** 를 강제할 수 있습니다. 실수로든 AI 든 main 에 직접 push 하거나 force push 하는 걸 막아줍니다.
→ Settings → **Branches** → Add rule (또는 Rulesets) → `main` → ✅ Require a pull request before merging

⌨️ CLAUDE.md 에 Git 규칙 적어두기

```
## Git 규칙
- main 브랜치에서 직접 작업하지 않는다. 작업 전 feat/이름 브랜치를 만든다
- 커밋 메시지: "타입: 설명" (feat/fix/style/docs/refactor/chore)
- 한 커밋에는 한 가지 일만
- 커밋 전에 git diff --stat 결과를 사용자에게 보여준다
- .env 와 비밀 정보는 절대 커밋하지 않는다
- 금지: git push --force, git reset --hard, git clean -fd, git branch -D
  (꼭 필요하면 이유를 설명하고 사용자 허락을 받는다)
```

📦 더 알고 싶은 사람만 (몰라도 실습 가능)

위험 명령을 아예 못 치게 막기. 클로드 코드 설정 파일 `.claude/settings.json` 의 `permissions.deny` 에 적어두면, 클로드가 그 명령을 실행하려 할 때 **자동으로 거부**됩니다.

```
{
  "permissions": {
    "deny": [
      "Bash(git push --force:*)",
      "Bash(git push -f:*)",
      "Bash(git reset --hard:*)",
      "Bash(git clean -fd:*)"
    ]
  }
}
```

CLAUDE.md 는 **"부탁"**, settings.json 의 deny 는 **"잠금장치"** 입니다.

⚠️ 자주 하는 실수

- 클로드의 "Do you want to proceed?" 에 **읽지 않고** Yes 연타
- "자동 승인" 모드로 켜두고 자리 비우기 → 최소한 **브랜치에서**, **커밋 후에** 맡기기
- 에러가 나자 클로드가 `--force` 로 해결하려 함 → **NO** 누르고 "force 없이 해결해줘"

💬 클로드에게 이렇게 말하기

```
CLAUDE.md 에 "Git 규칙" 섹션을 추가해줘. main 직접 작업 금지, 커밋 메시지 형식,
커밋 전 diff 보여주기, .env 커밋 금지, force push / reset --hard / clean -fd 금지.
```

```
push 가 거절됐어. --force 는 쓰지 말고, 왜 거절됐는지 설명한 다음 안전하게 해결해줘.
```

보너스

GitHub Actions 맛보기 (자동 검사 · 자동 배포)

📖 이게 뭐야?

GitHub Actions GitHub 이 빌려주는 **남의 깨끗한 컴퓨터**에서, push 나 PR 이 생길 때마다 **정해둔 일을 자동으로** 실행해주는 기능입니다.

CI (Continuous Integration) = PR 마다 자동으로 **검사** (테스트, 문법 검사) **CD (Continuous Deployment)** = main 에 합쳐지면 자동으로 **배포**

설정은 `.github/workflows/이름.yml` 파일 하나로 합니다.

```
# .github/workflows/check.yml — PR 마다 HTML 파일이 있는지 간단 검사
name: check
on:
  pull_request:
    branches: [main]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: test -f index.html && echo "index.html OK"
```

PR 화면 아래에 ✅ / ❌ 가 뜹니다. 14단계의 브랜치 보호에서 **"이 검사를 통과해야만 merge 가능"** 으로 묶을 수 있습니다.

💬 클로드에게 이렇게 말하기

```
.github/workflows/ci.yml 을 만들어줘. main 으로 가는 PR 마다 ubuntu 에서
npm ci → npm run lint → npm test 를 실행하게. 각 줄이 무슨 뜻인지 주석도 달아줘.
```

```
이 Vite 프로젝트를 GitHub Actions 로 빌드해서 GitHub Pages 에 배포하는 workflow 를 만들어줘.
vite.config 의 base 경로도 저장소 이름에 맞게 설정해줘.
```

부록 A

상황별 명령어 치트시트

하고 싶은 것	명령
지금 어디? / 뭐 있어?	`pwd` / `ls -a`
저장소 시작	`git init`
상황 확인 (수시로!)	`git status`
바뀐 내용 보기	`git diff` / `git diff --staged` / `git diff --stat`
상자에 담기	`git add 파일` / `git add .`
세이브	`git commit -m "타입: 설명"`
기록 보기	`git log --oneline --graph --all`
수정 취소 (add 전)	`git restore 파일`
상자에서 빼기	`git restore --staged 파일`
방금 커밋 취소 (push 전)	`git reset --soft HEAD~1`
push 한 커밋 취소	`git revert 커밋번호`

하고 싶은 것	명령
잠깐 치워두기 / 꺼내기	`git stash` / `git stash pop`
브랜치 만들고 이동	`git switch -c feat/이름`
브랜치 이동 / 목록	`git switch 이름` / `git branch`
합치기 / 취소	`git merge 이름` / `git merge --abort`
GitHub 로그인	`gh auth login`
GitHub 저장소 만들고 올리기	`gh repo create 이름 --public --source=. --remote=origin --push`
올리기 / 받아오기	`git push` / `git pull`
새 브랜치 처음 올리기	`git push -u origin 브랜치`
복사해 오기	`git clone 주소`
PR 만들기 / 목록 / 합치기	`gh pr create` / `gh pr list` / `gh pr merge 번호 --squash --delete-branch`
이슈 만들기 / 목록	`gh issue create` / `gh issue list`
브라우저로 저장소 열기	`gh repo view --web`

부록 B

에러 메시지 번역표

화면에 뜬 말	쉬운 말	해결
`command not found: git`	Git 이 없거나 터미널이 모름	설치 후 **터미널 새로 열기**
`fatal: not a git repository`	여기 저장소 아님	`pwd` 로 위치 확인 / `git init`
`Author identity unknown`	누군지 모름	`git config --global user.name/email`
`nothing to commit, working tree clean`	저장할 게 없음 (정상)	
`nothing added to commit but untracked files present`	상자가 비었음	`git add` 먼저
`pathspec '...' did not match any files`	그런 파일 없음	파일 이름·위치 확인 (Tab 자동완성!)
`src refspec main does not match any`	올릴 커밋 없음	커밋 먼저
`Permission denied` / `403`	권한·로그인 문제	`gh auth status`, 협업자 수락 확인
`[rejected] ... (fetch first)`	GitHub 에 내가 없는 커밋 있음	`git pull` → `git push`
`Your local changes ... would be overwritten`	저장 안 한 변경이 있어서 이동 불가	커밋 또는 `git stash`
`CONFLICT (content): Merge conflict in ...`	같은 줄을 둘 다 고침	10단계 4단계 해결법

화면에 뜬 말	쉬운 말	해결
`You are in 'detached HEAD' state`	과거 구경 중	`git switch main`
`LF will be replaced by CRLF`	줄바꿈 자동 변환 안내	**무시해도 됨**
vim 화면 (`~` 가 줄줄이)	편집기에 갇힘	`Esc` → `:q!` → Enter
`(END)` **또는** `:` **에서 멈춤**	긴 화면 보는 중	`q`

부록 C

바이브 코더의 Git 10계명

1. **작업 전** `git status`**, 작업 후** `git diff`
2. 클로드에게 큰 일 시키기 전에 **커밋하고 브랜치부터**
3. **main 에서 직접 작업하지 않는다**
4. 커밋은 **작게, 자주**, 메시지는 `타입: 설명`
5. `.gitignore` 는 **첫 커밋 전에**, `.env` 는 **절대** 올리지 않는다
6. 아침엔 `git pull`, 퇴근 전엔 `git push`
7. 충돌은 에러가 아니라 **질문**이다. 막히면 `git merge --abort`
8. push 한 기록은 **revert**, push 안 한 기록만 reset
9. `--force`, `--hard`, `clean -fd` 가 보이면 일단 NO
10. 클로드의 "Do you want to proceed?" 는 **읽고** 누른다